Waist-Only Underwear Sizing

Complete change guide for the **waist_only** branch

A precise comparison with the original four-measurement estimator documented in **docs/CODEBASE_GUIDE.pdf**. This guide covers implementation, behavior, API impact, tests, calibration, operating constraints, ambiguity, migration, and rollback.

Repository: wackademia/dimensions-check
Branch: waist_only
Generated: 22 September 2026

Document map

Section	Purpose
1. Executive summary	What changed and what did not
2. Baseline and scope	The exact original design used for comparison
3. Behavioral contract	Inputs, validation, matching, and outputs
4. Algorithm comparison	Before and after logic in detail
5. Reference ranges	Tolerance intervals, gaps, and overlaps
6. File-by-file changes	Implementation, UI, tests, and documentation
7. Pipeline impact	What remains unchanged in camera and CV processing
8. Verification	Automated tests and acceptance checks
9. Risks and operations	Limitations of deciding size from waist alone
10. Migration and rollback	How to deploy or reverse the branch
Appendix	Payload examples and decision cases

1. Executive summary

The original underwear estimator accepted a size only when **four calibrated flat-garment measurements** - front rise, full length, hips, and waist - were all within tolerance of exactly one supplied size reference. The waist_only branch changes that decision rule so that **only the calibrated flat waist width** determines M, L, XL, or 2XL.

> The change is intentionally narrow. Segmentation, landmark detection, inspection mask checks, landmark geometry checks, calibration, units, camera capture, concurrency protection, snapshots, and the full reference table remain in place.

Question	Original	waist_only
Required sizing inputs	4 measurements	Waist only
Decision distance	RMSE across 4 deltas	Absolute waist delta
Tolerance pass	Every delta within +/-0.50 in	Waist delta within +/-0.50 in
Ambiguity	Multiple full-vector matches	Overlapping waist intervals
Non-waist measurements	Affect the result	Ignored by estimate_size()
Unknown-size handling	Uncertain	Uncertain

2. Baseline and scope

2.1 Baseline document

The comparison baseline is **docs/CODEBASE_GUIDE.pdf**, especially Section 8.4, which documents the original four-dimensional `estimate_size()` algorithm. The baseline reference values are measured laid-flat garment dimensions in inches, not body measurements and not an industry-wide sizing chart.

2.2 In scope

- The size-decision function in src/garmentiq/underwear_sizes.py.
- Tests that define the sizing contract in test/test_underwear.py.
- Operator wording in phone_web/index.html.
- Sizing descriptions in docs/LIVE_PRODUCTION_LINE.md and docs/CODEBASE_GUIDE.md.

2.3 Explicitly out of scope

- No model retraining and no checkpoint changes.
- No landmark schema changes; shorts landmarks are still used for underwear.
- No change to camera capture, stillness detection, ROI cropping, or TLS hosting.
- No change to ruler/reference calibration or pixel-to-inch conversion.
- No change to frame rejection heuristics or request locking.
- No removal of non-waist values from REFERENCES or measurement responses.

> Important distinction: the application may continue to **measure and display** front rise, full length, and hips. They simply no longer influence the size returned by estimate_size().

3. Behavioral contract

3.1 Required input

The function now reads only `measurements['waist']`. Its distance must be a finite positive Python int or float, must not be bool, and its unit must be exactly `'in'`. Missing or invalid waist data produces an uncertain response.

3.2 Ignored input

Any values under `'front rise'`, `'full length'`, or `'hips'` are ignored by the estimator. They may be missing, invalid, or far from the reference without changing the size decision, provided waist remains valid.

3.3 Output compatibility

The response keys remain compatible: status, size, reason, tolerance_in, candidates, reference, closest_size, closest_reference, closest_deltas_in, and closest_rmse_in. The semantic difference is that each candidate's deltas_in now contains only waist, and rmse_in is the absolute waist delta rather than a four-dimensional RMSE.

3.4 Conservative acceptance remains

A nearest reference is still not automatically accepted. Exactly one reference must be within tolerance. Zero matches and multiple matches remain uncertain and require manual review.

4. Algorithm comparison

4.1 Original four-measurement algorithm

For every expected measurement name, validate inches and a positive finite distance. For each size, calculate four deltas, calculate their RMSE, and accept the candidate only when every absolute delta is at most the tolerance.

```
for name in ('front rise', 'full length', 'hips', 'waist'): validate(measurements[name]) for size, reference in REFERENCES.items(): deltas = values - reference rmse = sqrt(mean(delta ** 2)) within = all(abs(delta) <= tolerance for delta in deltas)
```

4.2 Waist-only algorithm

Validate only the waist measurement. For each size, subtract the reference waist. The absolute delta is both the comparison distance and the tolerance test.

```
waist = validate(measurements['waist']) for size, reference in REFERENCES.items(): delta = waist - reference['waist'] distance = abs(delta) within = abs(delta) <= tolerance
```

4.3 Selection rule

Number of candidates within tolerance	Result
0	status=uncertain; size=None; closest candidate still reported
1	status=matched; size=that candidate
2 or more	status=uncertain; size=None; manual review required

5. Waist references and tolerance geometry

The branch retains the supplied flat-waist references and the provisional +/-0.50 inch allowance. The following intervals are inclusive.

Size	Reference waist (in)	Accepted interval (in)
M	13.69	13.19 to 14.19
L	14.62	14.12 to 15.12
XL	15.79	15.29 to 16.29
2XL	16.84	16.34 to 17.34

5.1 Ambiguous M/L overlap

M and L overlap from 14.12 through 14.19 inches. A waist inside that interval matches both references and therefore returns uncertain. This is expected and prevents a forced label.

5.2 Unmatched gaps

There is a 0.17-inch gap between L and XL (15.12 to 15.29) and a 0.05-inch gap between XL and 2XL (16.29 to 16.34). Values strictly inside those gaps match no reference and return uncertain.

5.3 Why the boundary test changed

The old M + 0.50 inch test becomes ambiguous because 14.19 inches falls inside both M and L. The test now uses M - 0.50 inches (13.19), which is an unambiguous inclusive boundary; one additional hundredth (13.18) correctly becomes uncertain.

6. File-by-file change inventory

File	Change	Runtime effect
src/garmentiq/underwear_sizes.py	Added SIZE_MEASUREMENT='waist'; validates one input; compares one delta; updated reasons/docstring	Waist alone decides size
test/test_underwear.py	Replaced four-dimension expectations with waist-only contract tests; moved invalid-input checks to waist	Prevents regression to four-factor sizing
phone_web/index.html	Changed operator explanation from all four dimensions to flat waist	UI matches server behavior
phone_server.py	Updated one explanatory comment to describe waist-only ambiguity	No runtime behavior change
docs/LIVE_PRODUCTION_LINE.md	Updated acceptance and uncertainty wording	Operating guide matches branch
docs/CODEBASE_GUIDE.md	Updated estimator steps and uncertainty definition	Technical baseline is no longer misleading
docs/WAIST_ONLY_CHANGES.pdf	New complete comparison document	Review and handover artifact

6.1 Reference table deliberately retained

REFERENCES still stores front rise, full length, hips, and waist. This preserves the supplied sample record, the UI reference table, snapshot comparisons, and future ability to restore multi-factor sizing. Only the estimator's decision vector changed.

6.2 Response field names deliberately retained

The field name rmse_in remains for API compatibility even though, for a one-dimensional vector, RMSE is mathematically equal to absolute error. Renaming it would require unnecessary frontend and consumer changes.

7. What did not change in the processing pipeline

Stage	Unchanged behavior
Phone capture	Browser captures the selected ROI at native pixel resolution
Calibration	Known reference converts pixel distances to inches
Segmentation	BiRefNet produces the garment mask
Landmark detection	Shorts landmark layout supplies underwear points
Frame safety	Mask and landmark geometry can reject unsuitable frames
Measurement	LiveDetector may calculate all garment dimensions
Concurrency	One inference request at a time; busy requests receive 429
Unknown handling	Uncertain remains a normal response, not a server error

The waist-only change occurs after calibrated measurements exist. It does not make calibration optional, and it does not turn pixel width into inches by itself.

8. Verification and acceptance evidence

8.1 Automated suite

The underwear and HTTP integration suite completed successfully: **21 tests passed**. It covers exact reference sizes, tolerance boundaries, ambiguous results, invalid waist values, closest-size reporting, frame rejection, real measurement math, API validation, concurrency, error recovery, reference delivery, and frontend assets.

8.2 Contract-specific checks

- Changing full length to an extreme value no longer changes an XL waist match.
- Invalid or missing waist still returns uncertain.
- A calibrated 14.62-inch waist still resolves to L in the integration fixture.
- The L/XL midpoint under a widened tolerance is ambiguous and is not forced.
- API response shape remains compatible with the existing phone server.

8.3 Suggested production acceptance test

Step	Expected observation
Calibrate against a known reference	Scale is finite and expressed as pixels per inch
Measure one physical sample per size	Waist is close to its supplied reference
Repeat each sample 10 times	Variation is recorded; ambiguous runs remain uncertain
Test overlap/gap values	No forced result where two or zero references match
Move or resize camera	Recalibration is required before trusting inches

9. Risks, limitations, and operating implications

9.1 Reduced evidence

A single width cannot distinguish garments that share a waistband width but differ in rise, hip shape, stretch, cut, or labeling. The branch intentionally trades discrimination for a simpler rule.

9.2 Flat width is not body circumference

The waist value is the laid-flat garment width. It is not automatically a body-waist value, a doubled circumference, or a universal retail size. Elastic tension and measurement procedure matter.

9.3 Calibration remains decisive

Scale error directly shifts the only deciding feature. With four features, one noisy dimension could cause rejection; with one feature, a biased waist can cause a wrong match. Keep the camera fixed, keep garment and reference in the same plane, and recalibrate after geometry changes.

9.4 Tolerance is provisional

+/-0.50 inches is not a validated manufacturing tolerance or statistical confidence interval. Do not widen it merely to reduce uncertain outcomes; wider bands increase overlap and ambiguity.

9.5 Recommended guardrails

- Continue using InspectionRejected gates before size estimation.
- Show uncertainty to the operator rather than silently choosing closest_size.
- Record measured waist, delta, calibration, and image for audits.
- Validate references against real production samples before operational use.
- Treat a product/style change as a new reference-validation exercise.

10. Migration, deployment, and rollback

10.1 Deploying the branch

Fetch the repository and check out `waist_only`. Install the same runtime and model checkpoints used by main. No database migration, checkpoint conversion, or frontend build step is introduced by this change.

```
git fetch origin git switch waist_only python -m unittest discover -s test -p test_underwear.py -v
```

10.2 Rollback

Switch back to main to restore the original four-measurement estimator. Because the reference table and API response shape were preserved, rollback does not require data conversion.

```
git switch main
```

10.3 Review checklist

- Confirm the active branch is waist_only.
- Confirm CUDA/CPU runtime and checkpoints load successfully.
- Run the 21-test underwear suite.
- Verify the phone UI states that waist alone determines size.
- Perform physical repeatability testing before production decisions.

Appendix A - Example inputs and outputs

A.1 Matched L

```
Input: {'waist': {'distance': 14.62, 'unit': 'in'}, 'hips': {'distance': 99.0, 'unit': 'in'}} Decision:
{'status': 'matched', 'size': 'L', 'reason': 'Waist measurement matches one supplied reference.'}
```

The deliberately incorrect hips value is ignored. This demonstrates the new contract; it is not a recommendation to suppress measurement-quality checks elsewhere.

A.2 Ambiguous M/L

```
Input waist: 14.15 in M delta: +0.46 in (within tolerance) L delta: -0.47 in (within tolerance) Result:
uncertain because two candidates match
```

A.3 No match

```
Input waist: 15.20 in L delta: +0.58 in XL delta: -0.59 in Result: uncertain because no candidate is
within +/-0.50 in
```

A.4 Invalid calibration unit

```
Input: {'waist': {'distance': 500, 'unit': 'px'}} Result: uncertain - waist must be calibrated to inches
```

Appendix B - Source references

Repository source	Role in this guide
docs/CODEBASE_GUIDE.pdf	Original four-measurement design baseline
src/garmentiq/underwear_sizes.py	Sizing references, validation, and decision algorithm
test/test_underwear.py	Executable behavior contract and integration evidence
phone_web/index.html	Operator-facing explanation
docs/LIVE_PRODUCTION_LINE.md	Production operation and calibration guidance

End of document. This guide describes the waist_only branch only. It should be read together with the existing full codebase guide for architecture and module-level details.